UNIVERSITY of GREENWICH

COMP1549: Advanced Programming

Reflection

Dr Markus Wolf

19th March, 2026 - Week 10

Reflection

Reflection

UNIVERSITY of GREENWICH

- Reflection enables inspection of classes at runtime
- Makes it possible to construct instances of classes for which all you have is the name
- Once instantiated you can:
  - Access/modify the fields of the object
  - Invoke methods on the object
- It's also possible to access metadata (e.g. annotations, assembly attributes)

Reflection

UNIVERSITY of GREENWICH

- Using reflection, one can
  - Query objects for their abilities
  - Call their methods (invoke their methods) without knowing the method name, parameters and return type in advance

Why do we need it?

UNIVERSITY of GREENWICH

- Sometimes it is necessary to inspect compiled code at runtime
- Plug components together at runtime
- Factories can create new types
- Tools that analyse compiled code at runtime
  - A testing framework can reflect on compiled code, identify and execute the tests
  - A tool can reflect on a given assembly or byte code to establish the classes and their contents (e.g. for analysing complexity, statistics, comparing implementations, etc.)

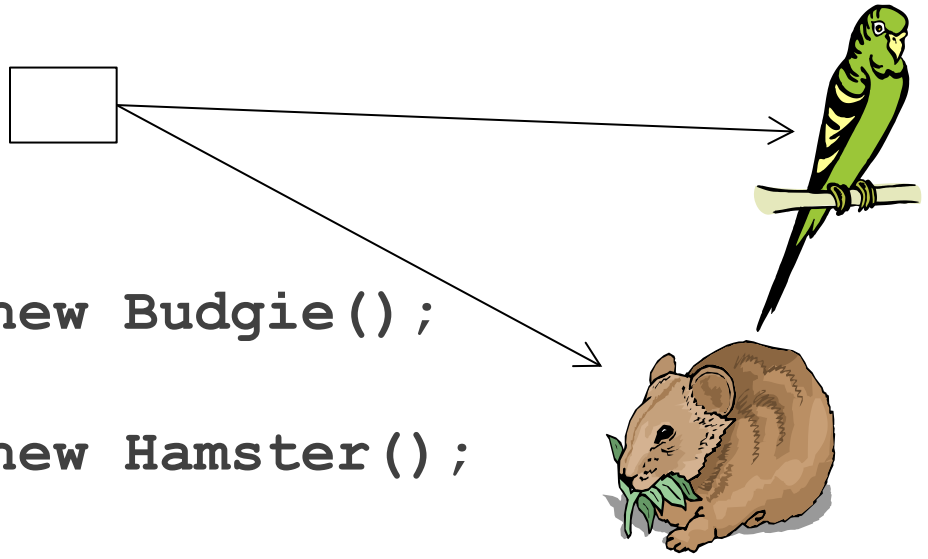
What is Exposed?

- Fields, constructors, methods, superclasses, annotations and implemented interfaces by name
- Using the Introspector and BeanInfo it will reveal properties, events and methods named or defined in accordance with the naming defined in the JavaBeans Specification

Runtime Type Information

- The type of a variable is known at compile time but the type of an object it refers to isn't fixed until runtime

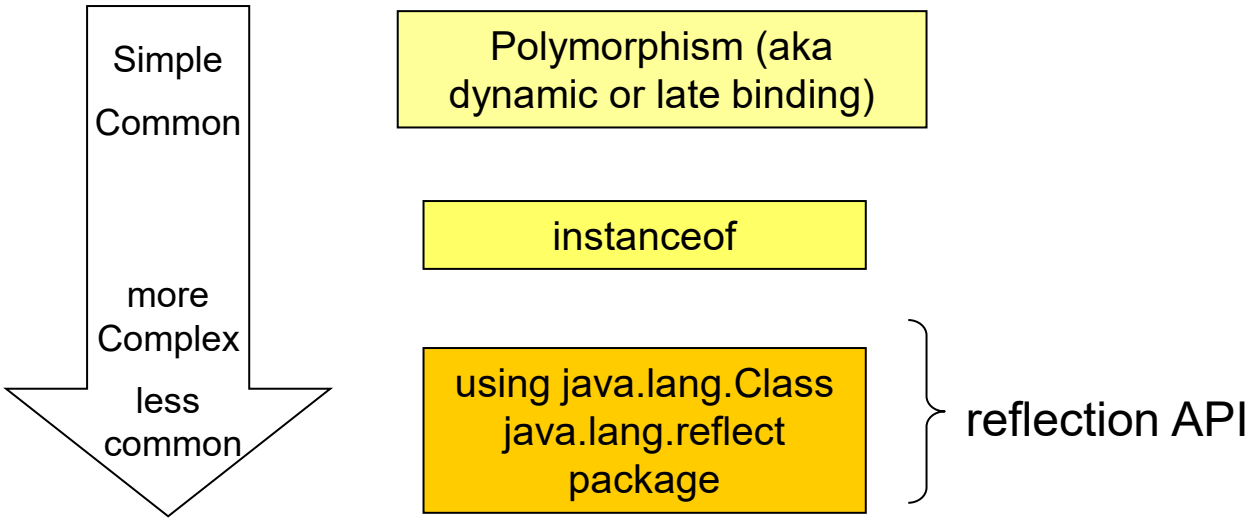
```
Pet  yourPet;
  …
if (age > 75)
      yourPet = new Budgie();
if (age < 12)
      yourPet = new Hamster();
```



- The type of the variable yourPet is fixed when you compile the code …
- …but the type of object referenced by the variable is decided according to the value of age at runtime

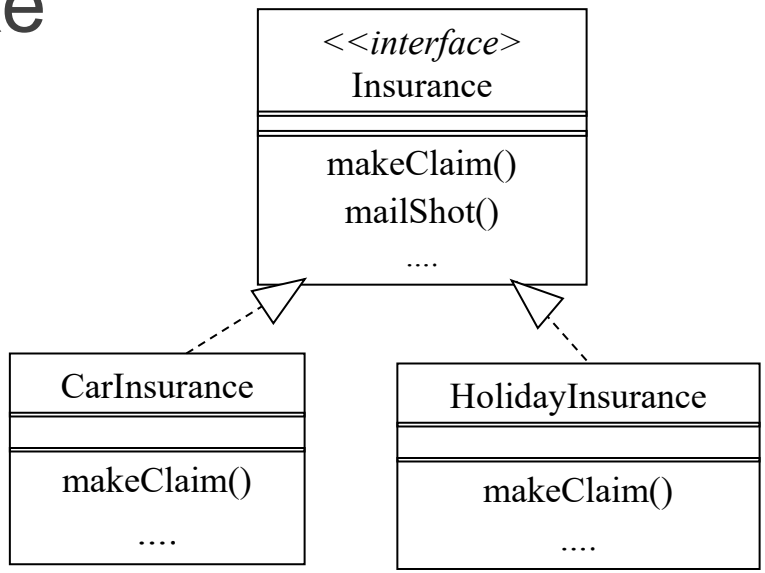
Runtime Type Information

- Before we look in detail at “reflection” itself we'll review other ways that type information is available to your program at runtime



Polymorphism

- We use it all the time to make our programs maintainable and extensible




```
List policies<Insurance> = new ArrayList<>();
...
for(Insurance policy : policies) {
        policy.makeClaim();
}
```

Correct method found at runtime

Finding the Type at Runtime Using `instanceof`

- Sometimes we need to test at runtime if an object belongs to a specific class
  - May want to call a method specific to it
  - May want to process objects of different classes differently
- Suppose we want only to mail shot holiday insurance holders

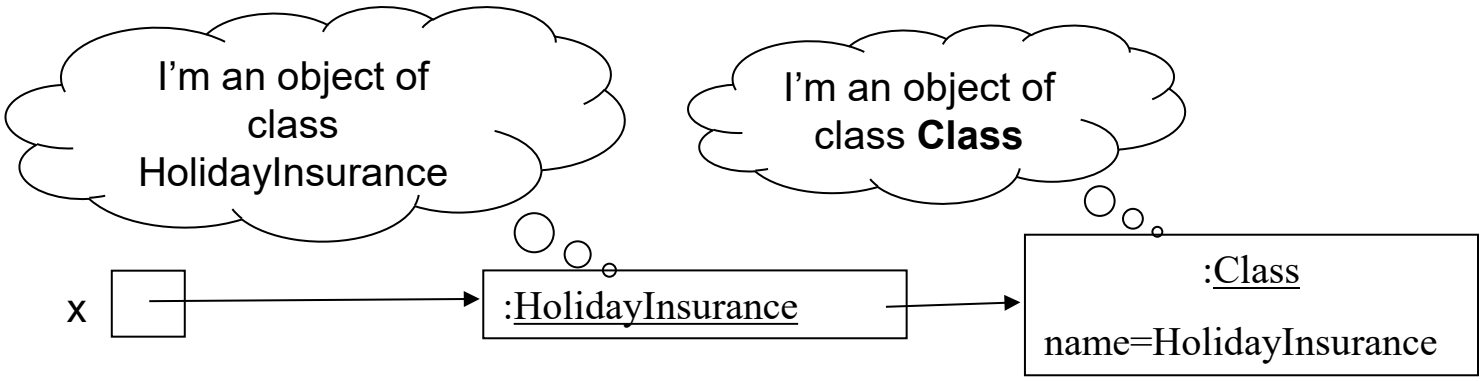
```
for(Insurance policy : policies) {
      if ( policy instanceof HolidayInsurance ) {
            policy.mailShot();
      }
}
```

Is this an instance of HolidayInsurance?

The Class `Class`

- instanceof is simple but limited
- The class java.lang.Class allows us to do more
- When a class is first loaded (e.g. when its first instance is created) an object of class Class is created representing the class

```
HolidayInsurance x = new HolidayInsurance();
```



The Class `Class`

- We can get hold of the class object and query it e.g.

```
for(Insurance policy : policies) {
  Class polClass = policy.getClass();
  String polClassName = polClass.getName();
  if (polClassName.equals("reflect.HolidayInsurance"))
  {
    policy.mailShot();
  }
}
```

This is the same functionality implemented using the Class class instead of instanceof

The Class `Class`

UNIVERSITY of GREENWICH

- Other ways for getting hold of an object of class `Class`
- Using a "class literal":
  - If the class name is known at compile time (say, Button), get class object

```
Class c = Button.class;
```

- Using the static Class method forName():
  - If class name is known (as a String variable str) at run time, get class object

```
Class c = Class.forName(str);
```

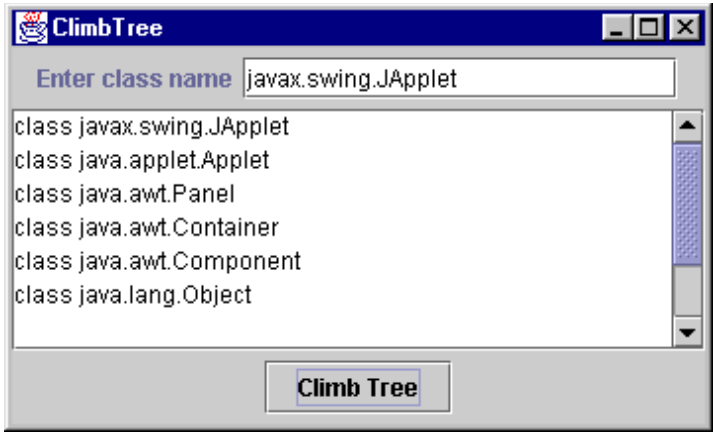
- If you have an object obj, get its class object with

```
Class c = obj.getClass();
```

Other Class `Class` Methods

- Class `Class` has numerous methods e.g. getSuperclass()
- Here is code to recursively climb up the class hierarchy from a given class

```
Class c = Class.forName(txtClass.getText());
goUpTree(c);
...
public void goUpTree(Class c) {
  txtOutput.append(c.toString() + "\n");
  Class superClass = c.getSuperclass();
  if (superClass != null)
    goUpTree(superClass);
}
```



Package `java.lang.reflect`

UNIVERSITY of GREENWICH

- Contains more classes that support reflection including:
  - class `Constructor`
  - class `Field`
  - class `Method`
- Allows
  - Instances to be created dynamically without using new
  - Methods to be called and fields (i.e. variables) to be accessed without the compiler knowing the class
- Widely used when writing tools (IDEs, plugins, documentation tools, testing tools, etc.)

The `Method` Class

UNIVERSITY of GREENWICH

- There are two ways to get Methods

  `getMethods();`

  - Returns all the methods for this class and any it inherits from super classes

  `getDeclaredMethods();`

  - Returns all the methods for this class only

- You can also get a specific method, but it takes more information

The `Method` Class

- Get a specific method

```
getMethod(String name, Class<?>... parameterTypes));
```

  - … means any number of parameters can be passed after the name (parameters here are a reference to the types of parameters the method takes)

- For example, say we have this method:

```
public int doSomething(String stuff, int times) {}
```

  - To get this specific method:

```
Class<?>[] paramTypes = {String.class, int.class};
getMethod("doSomething", paramTypes );
```

  - Types are directly passed because reflection will use the method "fingerprints " to track it down and return it to us

The `Method` Class

UNIVERSITY of GREENWICH

- Some other useful methods
  - **`getReturnType()`**
    - Gets the type of variable returned by this method
  - **`getParameterTypes()`**
    - Returns an array of parameters in the order the method takes them
  - **`invoke(Object obj, Object… args)`**
    - Calls/Runs this method on the given object, with parameters

The `Constructor` Class

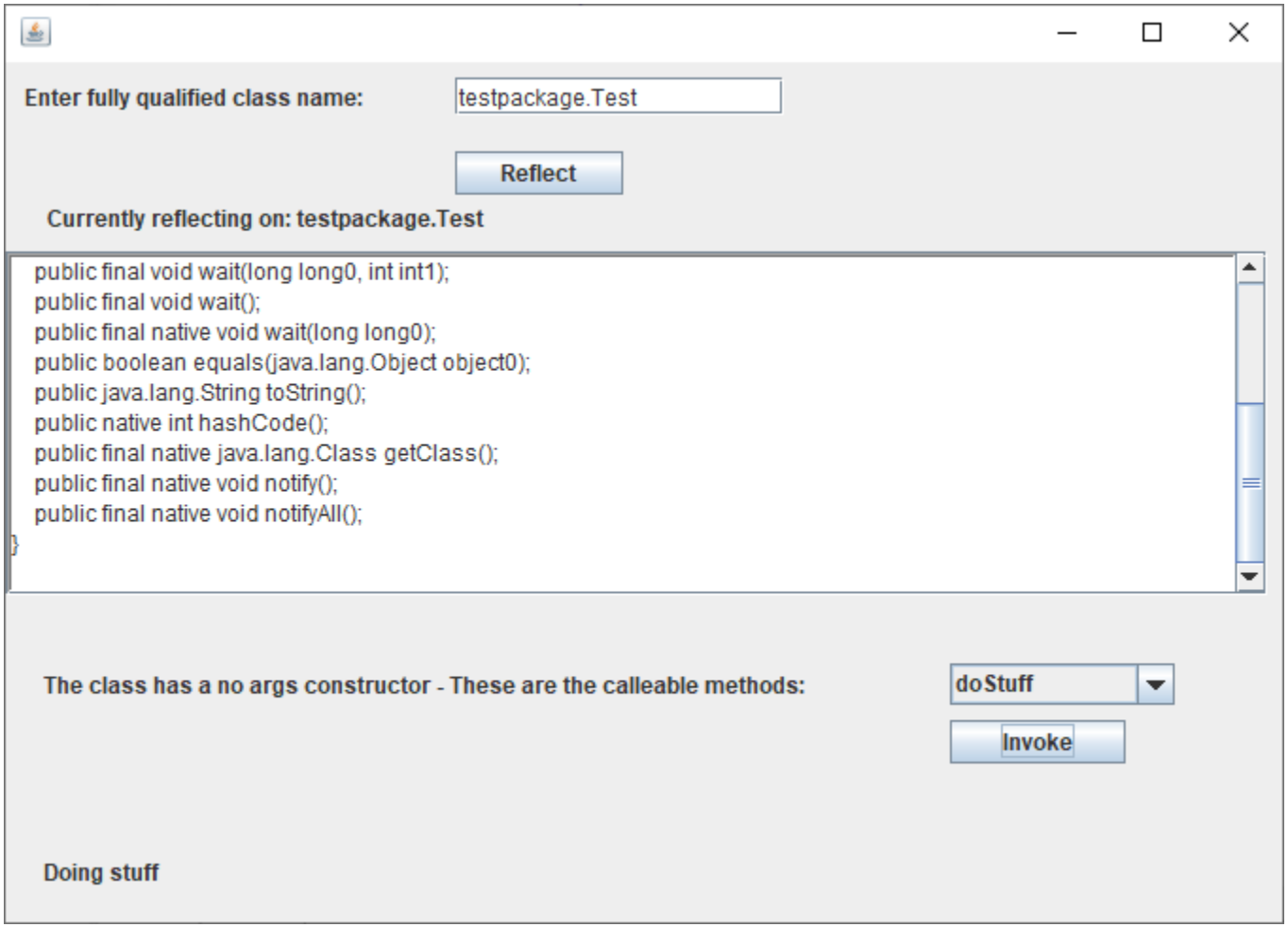
UNIVERSITY of GREENWICH

- Two ways to get Class constructors:
  - **`getConstructors()`**
    - Returns all public constructors for the class
  - **`getDeclaredConstructors()`**
    - Returns all constructors for the class
- Again, we can get specific constructors with:
  - **`getConstructor(Class<?>... parameterTypes));`**
    - Returns the constructor that takes the given parameters (i.e. parameter types).

An example

- A Reflector class has been implemented that allows you to expose the facilities of classes at runtime and invoke methods
- Reflector handles anything that might cause an exception
  - A lot can go wrong with reflective methods
- The user interface is implemented as a JFrame, which makes calls on the Reflector.java class

The Application



Getting hold of the class type

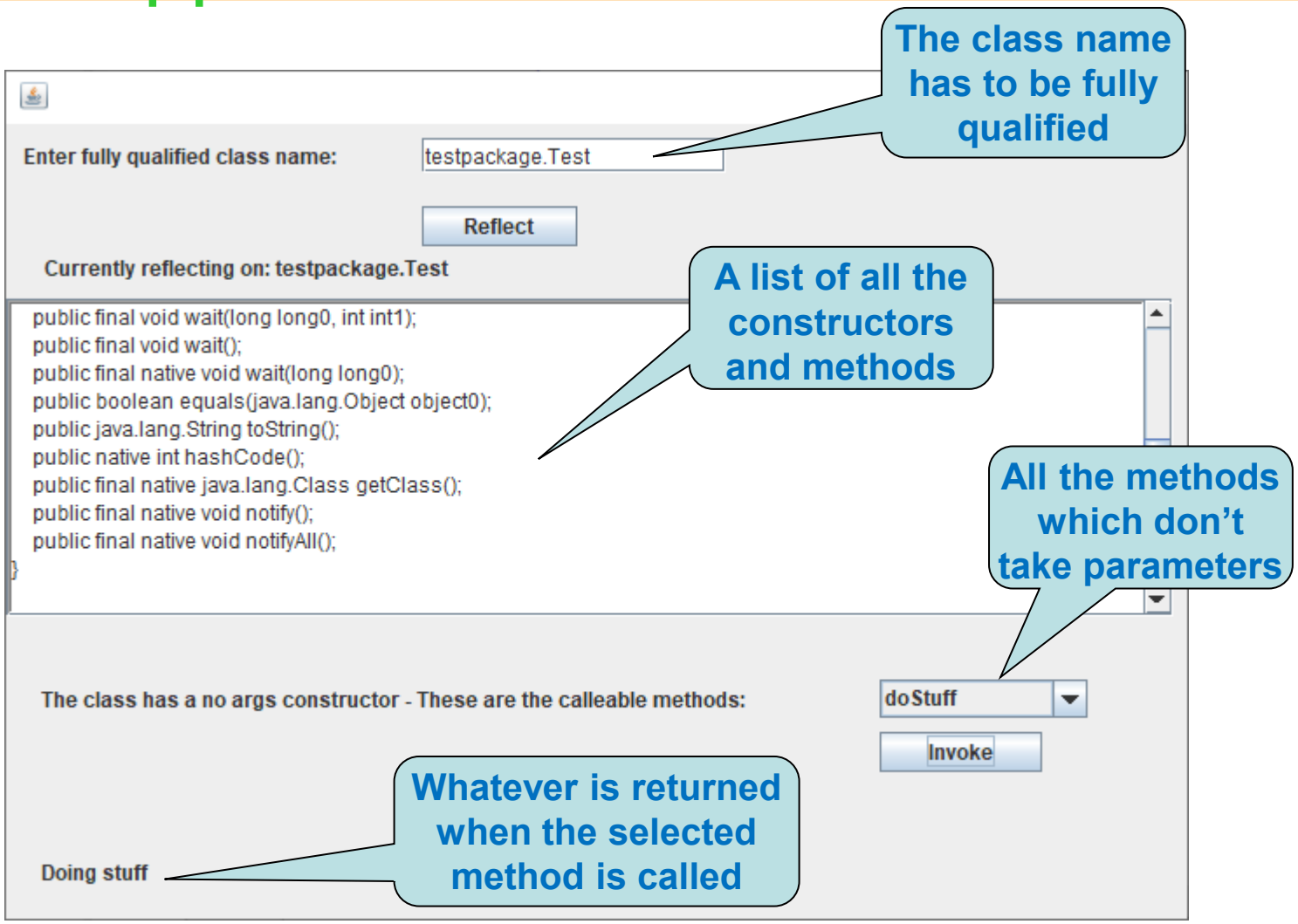
- The GUI calls through to the setClassName method of reflector
- Here's the Java code:

```
public void setClassName(String className)
{
   this.className = ((className == null)?"":className);
   this.className = this.className.trim();
   try
   {
      clazz =
   Thread.currentThread().getContextClassLoader().
   loadClass(className);
   }
   catch(ClassNotFoundException cnfe)
   { /* nothing here */ }
}
```

Loads the class identified by the string into the JVM

The file to reflect on has to be on the classpath

The Application



Getting Superclass Info

```
private StringBuffer getExtendsString(StringBuffer buf)
{

   if(clazz.getSuperclass() == Object.class
       ||clazz.getSuperclass()== null )
   {
       return buf;
   }

   buf.append("\n    extends " +
   clazz.getSuperclass().getName());
   return buf;
}
```

buf is the string that will be returned and displayed on the page – this will display all of the class details

Getting Modifers

UNIVERSITY of GREENWICH

- Java has an int based Enumeration with tests on the modifiers class

```
int mod = constructors[i].getModifiers();
if(Modifier.isPublic(mod))
```

- And you can do this

```
buf.append("\n    " + Modifier.toString(mod) + " ");
```

Method Information

- You can get method and constructor information in very similar ways

```
private StringBuffer getConstructorString(StringBuffer buf)
{
   Constructor[] constructors = clazz.getConstructors();
   if(constructors == null){ return buf; }
   for(int i = 0; i < constructors.length; i++)
   {
        buf.append("\n    " + Modifier.toString(mod) +
                        " " + constructors[i].getName() +"(");
        Class[] params = constructors[i].getParameterTypes();
        if(params.length > 0)
        {
                buf = getParameterString(buf, params);
        }
        buf.append(");");
   }
   return buf;
}
```

This is a method we defined in our Reflector class

We will need the list of Types of all parameters (the types are of type Class)

Parameter Information

```
private StringBuffer getParameterString(StringBuffer buf,
   Class[] params)
{
   buf.append(params[0].getName() + " " +
       lowerFirstChar(resolveClassName(params[0].getName())
   ) +       "0");
   for(int i = 1; i<params.length; i++)
   {
       buf.append(", ");
       buf.append(params[i].getName() + " " +

       lowerFirstChar(resolveClassName(params[i].getName())
     ) +     i);
   }
   return buf;
}
```

resolveClassName is a method implemented in the Reflector class that will take the fully qualified name and return only the name of the item (without package details)

ResolveClassName Method

UNIVERSITY of GREENWICH

```
private static String resolveClassName(
      String className)
{
     int index = className.lastIndexOf('.');
     if((index)>-1)
     {
          return className.substring(index+1);
     }
     else
     {
          return className;
     }
}
```

Checks for the last full stop in the fully qualified class name

Strips away the package by getting a substring from the last full stop onwards

Invoking Methods

UNIVERSITY of GREENWICH

```
return methods[i].invoke( target, new Object[0]);
```

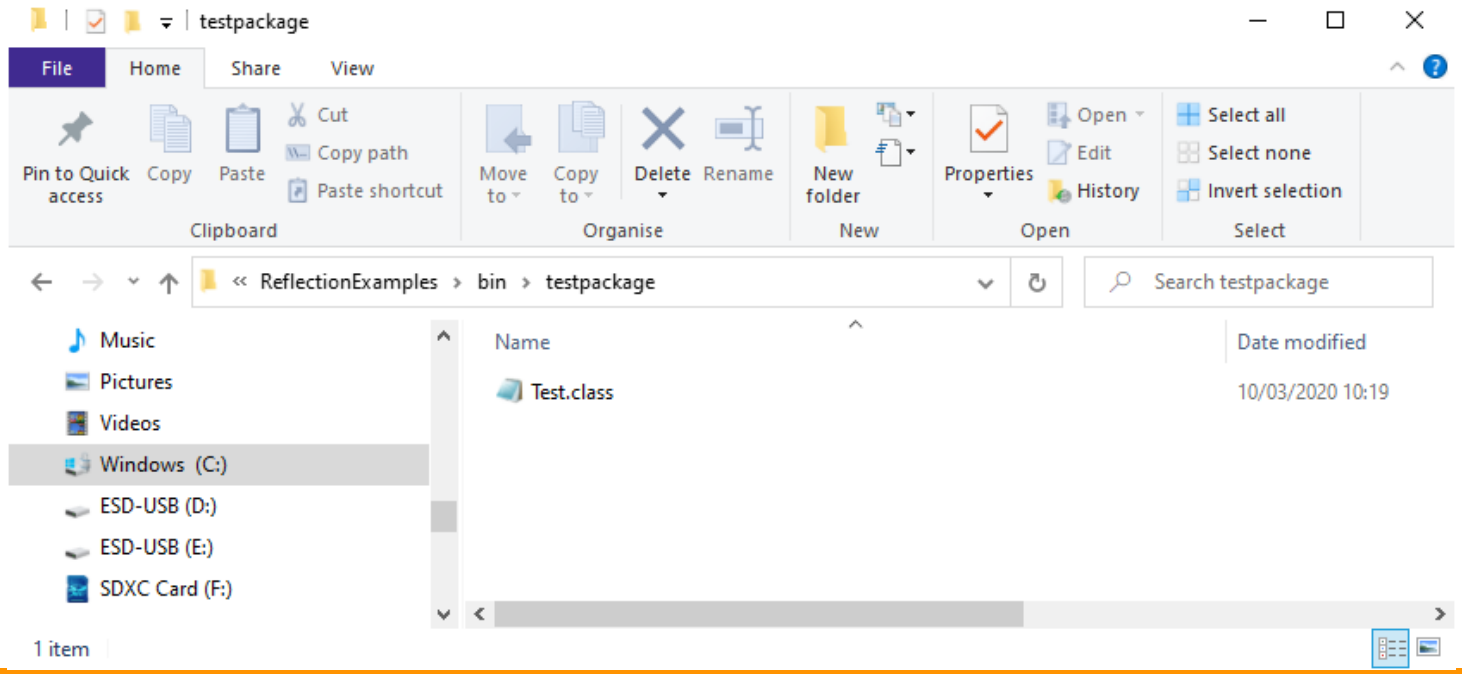
The object the method is invoked on

The parameters for the method

Don’t forget

- To invoke a method on a class using dynamic reflection, you must have the class somewhere in your path

- In the example, you need to put the compiled classes in the bin folder and they have to be placed in a folder structure that matches the package name



Reflection with Design Patterns

UNIVERSITY of GREENWICH

- Many of the object- oriented design patterns can benefit from reflection
- Reflection can extend the decoupling of objects that design patterns offer
- Can simplify design patterns, e.g.:
  - Factory
  - Observer (i.e. Publish/Subscribe)
    - E.g. to subscribe to events at runtime

Factory Without Reflection

UNIVERSITY of GREENWICH

```
public static Shape getShapeFactory(String sp) {
    Shape shapeObj = null;
    if (sp.equals("Circle")
        shapeObj = new Circle();
    else if (sp.equals("Square")
        shapeObj = new Square();
    else if (sp.equals("Triangle")
        shapeObj = new Triangle();
    // continue for each shape manufactured
    return shapeObj;
}
```

Factory With Reflection

UNIVERSITY of GREENWICH

```
public static Shape getShapeFactory(String sp) {
    Shape shapeObj = null;
    try {
      shapeObj = (Shape)Class.forName(sp).newInstance();
    } catch (ClassNotFoundException e) {
      ...
    }
    return shapeObj;
}
```

Reflection is Powerful

UNIVERSITY of GREENWICH

- Using reflection, you can
  - Convert strings into classes and objects at runtime
  - Ask detailed questions in code about the abilities of a type
  - Dynamically compile, load, and add classes to a running program

Disadvantages of Reflection

UNIVERSITY of GREENWICH

- Reflection should not be used in “normal” programming where we already have access to the classes and interfaces because:
  - Performance overhead
    - Since reflection resolves types dynamically, it involves processing like scanning the class path to find the class to load, which could cause slow performance
  - High Maintenance
    - Reflection code is harder to read, understand and debug
    - Reflection code is more verbose
    - Any issues with the code can’t always be found at compile time - avoiding compile time error checking increases likelihood of runtime exceptions
    - Extensive use in "normal" application programming tends to suggest design flaw

Uses of Reflection

UNIVERSITY of GREENWICH

- Creating tools (IDEs, testing etc.)
- Enabling very flexible systems
  - E.g. write any object to a database by dynamically creating the SQL
- Dynamic systems - perhaps where classes are added while a system is running
- Used internally by several APIs
  - e.g. JavaBeans and RMI

Interface as an Alternative

- Interfaces are sometimes a better alternative to reflection
- Interfaces still enable developers to write code that uses classes which may not have been implemented yet

Reflection and Modules

UNIVERSITY of GREENWICH

- We looked at modules in the lecture on components
- One of the feature of modules is the ability to restrict access via reflection
- Reflection lets us view the structure of a compiled class, instantiate it and invoke its methods (even when they are private)
  - Modules can't restrict reflection on a class' structure
  - Modules can restrict the instantiation of objects and invocation of methods

Modules – Exports

- We have already seen that when using Java modules, you can control which packages are accessible to other modules
  - E.g. `exports some.pkg;`
- It is also possible to do a qualified export and export only to one or more specific modules
  - E.g. `exports some.pkg to ExtModule;`
- If a package is exported, using reflection we can see its structure, instantiate it and invoke public methods

Modules - Opens

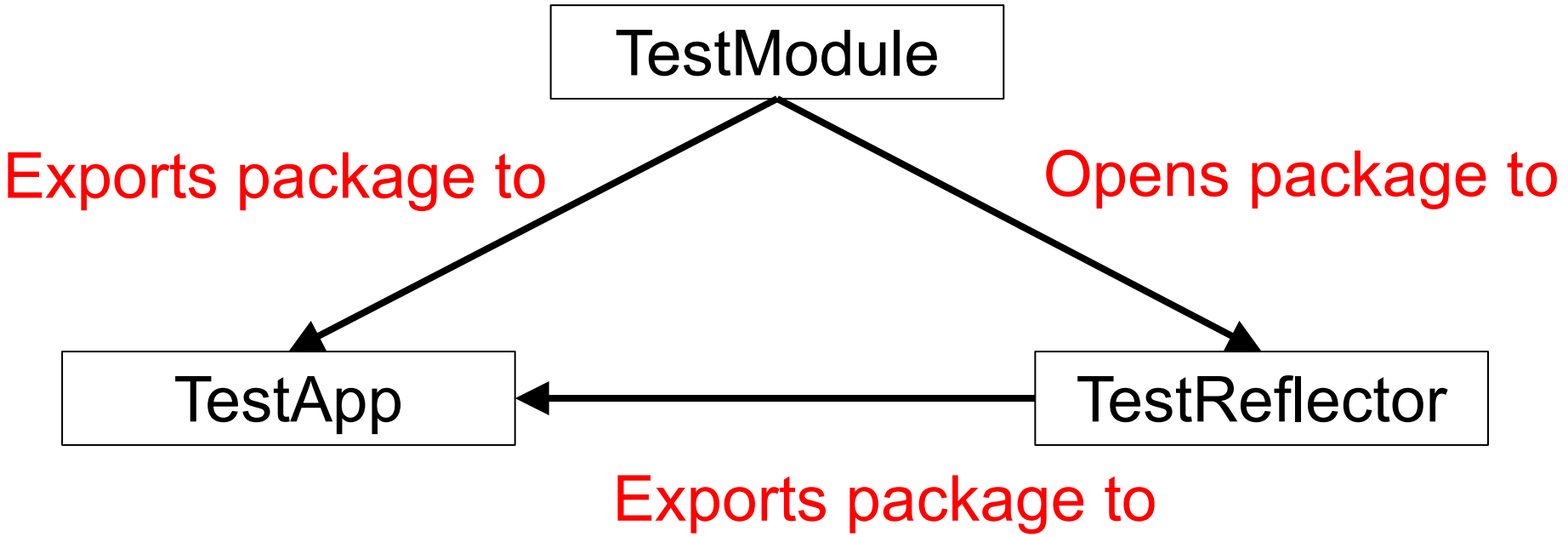
- It is possible to open a module or package for complete reflective access
- There are three options:
  - Open the module (automatically opens all contained packages)
  - Open a specific package to any module
  - Open a specific package to one or more specific modules

Modules - Opens

- Open the module
  - `open module MyModule {}`
- Open a specific package to any module
  - `opens some.pkg;`
- Open a specific package to one or more specific modules
  - `opens some.pkg to ExtModule;`

Module Reflection Example

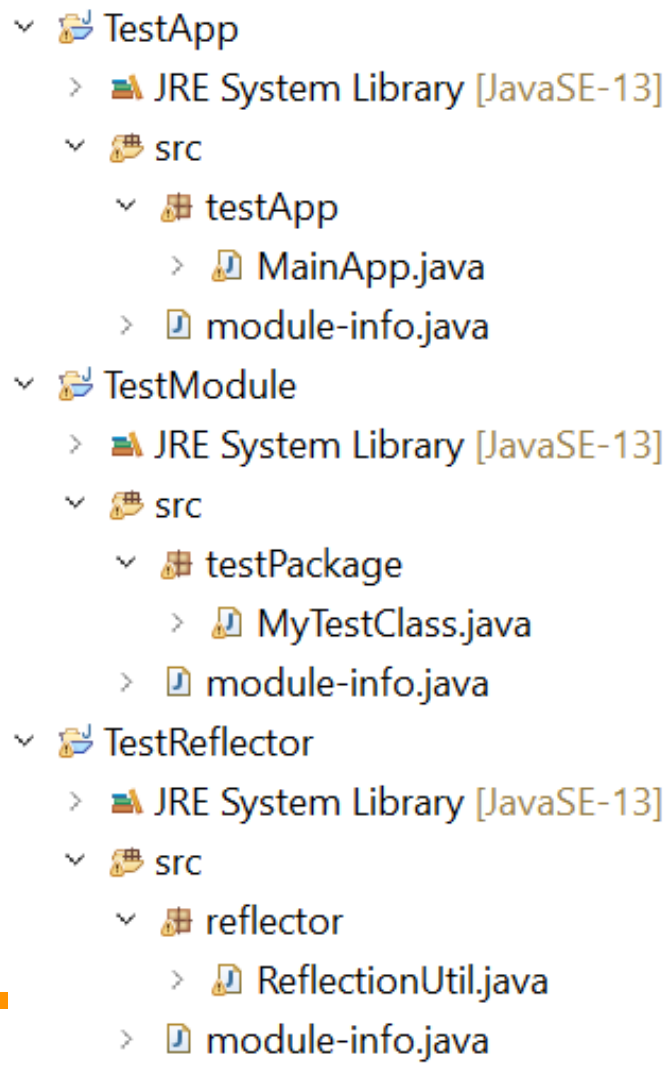
- Let's have a look at an example using three separate projects



Module Reflection Example

UNIVERSITY of GREENWICH

- Here is an overview of the three projects:



Module Reflection Example

UNIVERSITY of GREENWICH

- TestModule
  - Contains a package containing a class with a public and a private method
  - Exports the package to TestApp
  - Opens the package to TestReflector
- TestReflector
  - Contains a package with a utility method which reflects on a given class, invokes its methods and returns a list of all methods contained within it
  - Exports the package to TestApp

Module Reflection Example

UNIVERSITY of GREENWICH

- TestApp
  - Contains a package with a static void main method which loads the class from the TestModule project, passes it to the TestReflector module and prints the methods returned
  - Requires the TestModule
  - Requires the TestReflector

Module Reflector Example

UNIVERSITY of GREENWICH

- The content of the three module descriptors

```
module TestModule {
    opens testPackage to TestReflector;
    exports testPackage to TestApp;
}
```

```
module TestReflector {
    exports reflector;
}
```

```
module TestApp {
    requires TestModule;
    requires TestReflector;
}
```

Module Reflection Example

UNIVERSITY of GREENWICH

- Code in the MyTestClass (TestModule)

```
package testPackage;

public class MyTestClass {

    public void publicTestMethod() {
        System.out.println("Running public test");
    }

    private void privateTestMethod() {
        System.out.println("Running private test");
    }
}
```

Module Reflection Example

UNIVERSITY of GREENWICH

- Code in the ReflectionUtil class (TestReflector)

```
package reflector;

import java.lang.reflect.*;
import java.util.ArrayList;

public class ReflectionUtil {
    public static ArrayList<String> getMethods(Class theClass)
        throws IllegalAccessException,
               InvocationTargetException,
               InstantiationException {

            //Content of the method is on the next slide

    }
}
```

Declares that it throws a number of exceptions which can occur during reflection

Module Reflection Example

UNIVERSITY of GREENWICH

- Code in the getMethods method

```
Method[] declaredMethods = theClass.getDeclaredMethods();
ArrayList<String> names = new ArrayList<String>();
Object theObject =
    theClass.getDeclaredConstructors()[0].newInstance();

for (Method declaredMethod : declaredMethods) {
    names.add(declaredMethod.getName());
    declaredMethod.setAccessible(true);
    declaredMethod.invoke(theObject, null);
}
return names;
```

setAccessible() is called to suppress the check for access control

Module Reflection Example

UNIVERSITY of GREENWICH

- Code in the MainApp class (TestApp)

```
package testApp;

import java.lang.reflect.InvocationTargetException;
import java.util.ArrayList;

import reflector.*;

public class MainApp {
public static void main(String[] args) throws IllegalAccessException,
ClassNotFoundException, InvocationTargetException,
InstantiationException {
        Class obj = Class.forName("testPackage.MyTestClass");
        ArrayList<String> names = ReflectionUtil.getMethods(obj);
        System.out.println(names);
    }
}
```

Module Reflection Example

- This is what we see at runtime

```
Running private test
Running public test
[privateTestMethod, publicTestMethod]
```

- This is what you see if you don't open the package for reflection

```
module TestModule {
    //opens testPackage to TestReflector;
    exports testPackage to TestApp;
}
```

```
Exception in thread "main" java.lang.IllegalAccessException: class reflector.ReflectionUtil (in module TestReflector)
        at java.base/jdk.internal.reflect.Reflection.newIllegalAccessException(Reflection.java:385)
        at java.base/java.lang.reflect.AccessibleObject.checkAccess(AccessibleObject.java:693)
        at java.base/java.lang.reflect.Constructor.newInstanceWithCaller(Constructor.java:490)
        at java.base/java.lang.reflect.Constructor.newInstance(Constructor.java:481)
        at TestReflector/reflector.ReflectionUtil.getMethods(ReflectionUtil.java:10)
        at TestApp/testApp.MainApp.main(MainApp.java:11)
```

UNIVERSITY of GREENWICH

End of week 10!